Giriş və Tapşırıq 1: Node sinfinin yaradılması

Bu laboratoriya işində xətti bağlı siyahıların (`Singly Linked List`) əsas mexanizmləri, elementlərin əlavə olunması, silinməsi, axtarışı və siyahı üzərində müxtəlif əməliyyatların yerinə yetirilməsi, həmçinin obyektyönümlü proqramlaşdırma prinsipləri tətbiq edilmişdir.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def display(self):
        current = self.head
        res = []
        while current:
            res.append(str(current.data))
            current = current.next
        print(" -> ".join(res) + " -> None")
```

Tapşırıq 2, 3 & 4: Əlavəetmə Əməliyyatları

Siyahının sonuna (`append`), əvvəlinə (`prepend`) və verilmiş elementdən sonrakı mövqeyə (`insert_after`) yeni qovşaqların əlavə edilməsi metodları aşağıdakı kimi tərtib olunmuşdur:

```
    def append(self, data):
        yeni = Node(data)
        if not self.head:
            self.head = yeni
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = yeni

    def prepend(self, data):
        yeni = Node(data)
        yeni.next = self.head
        self.head = yeni

    def insert_after(self, hansi_noddan_sonra, data):
        current = self.head
        while current:
            if current.data == hansi_noddan_sonra:
                yeni = Node(data)
                yeni.next = current.next
                current.next = yeni
                return
            current = current.next
```

Tapşırıq 5, 6 & 7: Silmə, Axtarış və Uzunluq

Siyahıdan müəyyən dəyərə malik qovşağın silinməsi (`delete_node`), elementin axtarılması (`search`) və elementlərin ümumi sayının hesablanması (`length`) alqoritmləri:

```
    def delete_node(self, deyer):
        current = self.head
        if current and current.data == deyer:
            self.head = current.next
            current = None
            return
        prev = None
        while current and current.data != deyer:
            prev = current
            current = current.next
        if not current:
            return
        prev.next = current.next
        current = None

    def search(self, value):
        current = self.head
        while current:
            if current.data == value:
                print("Element tapıldı.")
                return True
            current = current.next
        print("Element tapılmadı.")
        return False

    def length(self):
        say = 0
        current = self.head
        while current:
            say += 1
            current = current.next
        return say
```

Tapşırıq 8 & 9: Maks/Min və Siyahının Tərs Çevrilməsi

Mövcud siyahı daxilində ən böyük və ən kiçik elementlərin tapılması, həmçinin yaddaşda əlavə köməkçi siyahı yaratmadan (`in-place`) istiqamətin tərsinə çevrilməsi (`reverse`) funksiyaları:

```
    def find_max_min(self):
        if not self.head:
            return None, None
        maks = self.head.data
        minn = self.head.data
        current = self.head.next
        while current:
            if current.data > maks: maks = current.data
            if current.data < minn: minn = current.data
            current = current.next
        return maks, minn

    def reverse(self):
        prev = None
        current = self.head
        while current:
            next_nod = current.next
            current.next = prev
            prev = current
            current = next_nod
        self.head = prev
```

Nəticə